

Saparali Adina [07-P]

Task 1.1 – Verification of Non-Determinism

The program was executed 10 consecutive times using a team of four threads. The observed logical rank order was 2, 1, 3, 0 rather than the sequential order 0, 1, 2, 3. In these experiments, the same ordering occurred in all ten runs. This does not mean that parallel execution has a guaranteed order. Thread execution is controlled by the operating system scheduler and the JVM runtime, so the order in which threads reach the output statement is not specified.

Task 1.2 – Thread Oversubscription Sweep

The thread team size was varied from 1 to 64 threads. The lowest measured execution time was 19.4502 ms with 4 threads. Execution time remained relatively similar for small thread counts, but increased when the number of threads reached 16, 32, and 64. This suggests that creating and managing additional software threads introduces scheduling and synchronization overhead. When the number of threads becomes large relative to available CPU resources, additional threads do not necessarily improve performance.

Threads P	Time, ms
1	20.6013
2	20.6306
4	19.4502
8	21.8518
16	24.1260
32	26.3690
64	25.0045

Threads	Time
1	0.5720 s
8	0.8548 s
16	1.1634 s

32	2.0642 s
----	----------

Task 1.3 – CPU Core Saturation Analysis

A CPU-intensive workload was added by performing repeated square-root calculations inside each thread. The test was executed with 1, 8, 16, and 32 threads. The system used an AMD Ryzen 7 5800HS processor with 8 physical cores and 16 logical processors. As the thread count increased, CPU activity was distributed across more logical processors. At 16 threads, the workload could utilize all available logical processors. At 32 threads, the number of software threads exceeded the available hardware threads, creating oversubscription. In this case, the operating system had to schedule multiple software threads on the same logical processors, increasing scheduling overhead and execution time.

1.1. Question: Why do thread IDs print in non-sequential order across runs? Which component controls this behavior?

Answer:

Thread IDs can appear in a non-sequential order because the threads execute in parallel. The operating system scheduler decides when each thread gets CPU time. Different threads may start, pause, or finish at different moments. Because of this, the output order is not guaranteed. In our experiment, the order was 2, 1, 3, 0 instead of 0, 1, 2, 3.

1.2. Question: What happens when the number of threads exceeds the number of physical CPU cores?

Answer:

Oversubscription happens when the program creates more software threads than the CPU can execute at the same time. Our processor has 8 physical cores and 16 logical processors. When we used 32 threads, the operating system had to share the available CPU resources between more threads. This creates context switching and additional scheduling overhead. It can also reduce cache efficiency because thread data may be replaced more often.

1.3. Question: What is the role of the implicit barrier at the end of a parallel region?

Answer:

A barrier makes all worker threads wait until every thread has completed its work. Only after all threads reach the barrier can the program continue to the next serial part. Without a barrier, one thread could continue while other threads are still working. This could cause incorrect results or access to incomplete data.

Question 1.4 — Hardware Thread vs OS Thread vs Virtual Thread

Answer:

A hardware thread is a CPU execution resource provided by the processor. For example, our AMD Ryzen 7 5800HS has 8 physical cores and 16 logical processors.

An operating system thread is a software execution unit managed by the OS scheduler.

The operating system assigns these threads to available CPU hardware threads.

A virtual or green thread is managed mainly by a language runtime instead of directly by the operating system. Many virtual threads can share a smaller number of operating system threads.

Lab 2:

Threads	Naive Race π	Error
1	3.141592653590	6.33e-13
2	1.557808765887	1.583784
4	0.979914652507	2.161678
8	0.388047046084	2.753546

Lab 2 Conclusion: The naive shared accumulator produced increasingly incorrect Pi values as the thread count increased because multiple threads performed unsynchronized read-modify-write operations. The critical section restored numerical correctness but introduced a 700.80% synchronization overhead. Parallel reduction provided both correct results and strong scalability. The measured speedup increased from 1.0× with one thread to 13.414× with sixteen threads, while efficiency decreased to 0.838 because of synchronization, scheduling, and hardware overhead.

Q2.1: A normal update consists of LOAD, ADD, and STORE operations. Two threads can load the same old value before either stores the new value. One update then overwrites the other, creating a lost update.

Q2.2: A critical section serializes updates through one shared lock. A reduction allows threads to calculate private partial results and combines them using a tree, which requires fewer synchronization stages and scales better.

Q2.3: With 5% strictly serial execution, Amdahl's Law gives a theoretical maximum speedup of **20×** even with infinitely many processors. Real performance may flatten earlier because of synchronization, memory bandwidth, cache effects, scheduling overhead, and contention.

Q2.4: Modern processors provide atomic instructions and cache-coherence mechanisms that make an update indivisible from the perspective of competing cores. The manual specifically asks about mechanisms such as MESI/cache locking and load-linked/store-conditional style primitives.

Lab 3 Conclusion:

The Mandelbrot benchmark showed that scheduling performance strongly depends on chunk size. Small chunks generally provided better execution times because work was distributed more evenly between threads. Large chunks, especially $C = 256$, caused severe load imbalance because only a small number of large work blocks were available. The best measured execution time was 87.488 ms using dynamic scheduling with 16 threads and chunk size 1. The results demonstrate the trade-off between scheduling overhead and load balancing.

Q3.1: Dynamic scheduling with chunk size 1 requires threads to access the shared work queue very frequently. This increases synchronization and atomic-operation overhead.

Q3.2: In static scheduling, threads assigned rows near the central and complex parts of the Mandelbrot set can become stragglers because these pixels require more iterations before escaping.

Q3.3: Guided scheduling begins with relatively large chunks and gradually decreases the chunk size as less work remains. This reduces scheduling overhead at the beginning while improving load balance near the end.

Q3.4: Static scheduling is suitable for uniform workloads. Dynamic scheduling is useful when iteration costs are highly irregular. Guided scheduling is useful when the workload is irregular but dynamic scheduling with very small chunks would create too much overhead.

